Hardware for Deep Learning

Systems, Chips, Algorithms

Dr. Rene Mueller rene.mueller@huawei.ch

Expert Researcher – Computer Systems, Huawei Technologies Switzerland AG

Zurich Office

1. Computing architecture, cover the almost all aspects of computing

Von Neumann Lab

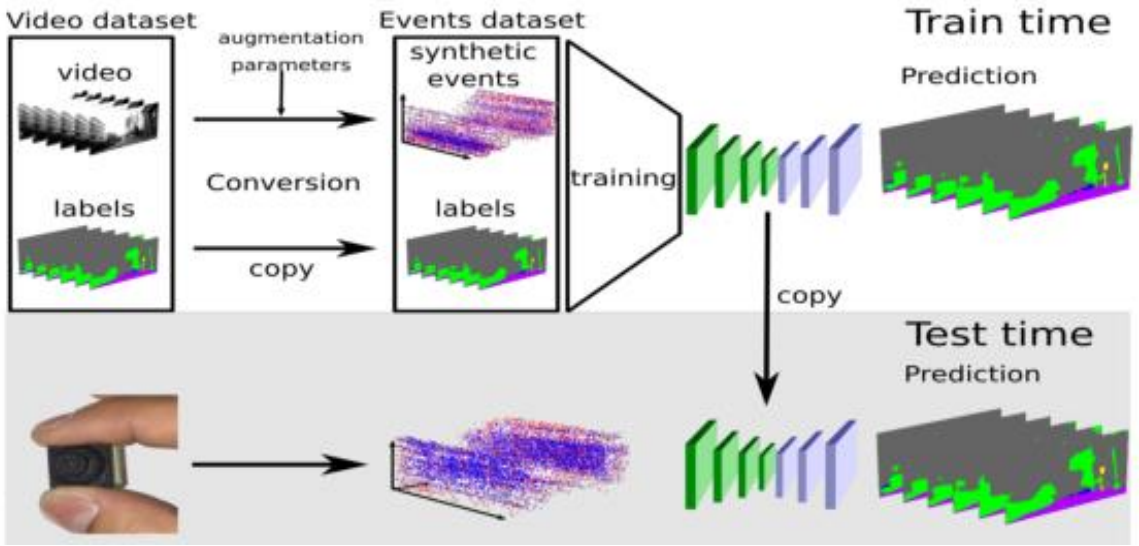
- **Tools and Methodology**
  - Hardware design automation
  - Design Space Exploration
- **Compiler and Software and hardware co-design**
  - Parallelism exploration and DSA mapping
  - Hardware Software Co-Design
- **System structure (HPC)**
  - Performance analysis and improvement
- **Instruction Set**
  - ISA definition and verification(Formal)
- **Microarchitecture**
  - CPU/GPU micro-architecture study for performance and power efficiency
- **Hardware (chip) security**
  - Security architecture study
  - Security beyond Isolation
- **Computing Theory**
  - Basic computing theory study
  - New computing model study

1. Including processor micro-architecture, SoC chip architecture, modeling simulation and verification, chip agile development (including EDA), open source ecology, compiler and tool chain and optimization, system software, security and other computer architecture full-scale cooperation. Short-term key breakthroughs; long-term targeted institutional research.

2. Talent building and training (PhD, postdoc, academic vacation), including Dr. Funding, postdoc. Post-doctoral, scientists and researchers.

Hiring Interns at PhD and Master level:
https://www.linkedin.com/jobs/search/?f_E=1&keywords=huawei

2. Computing Vision:
New algorithm for next generation Tech with partner.



HUAWEI

Outline

1) Quantization

2) Huawei DaVinici Core and Ascend SoC

Number Representations

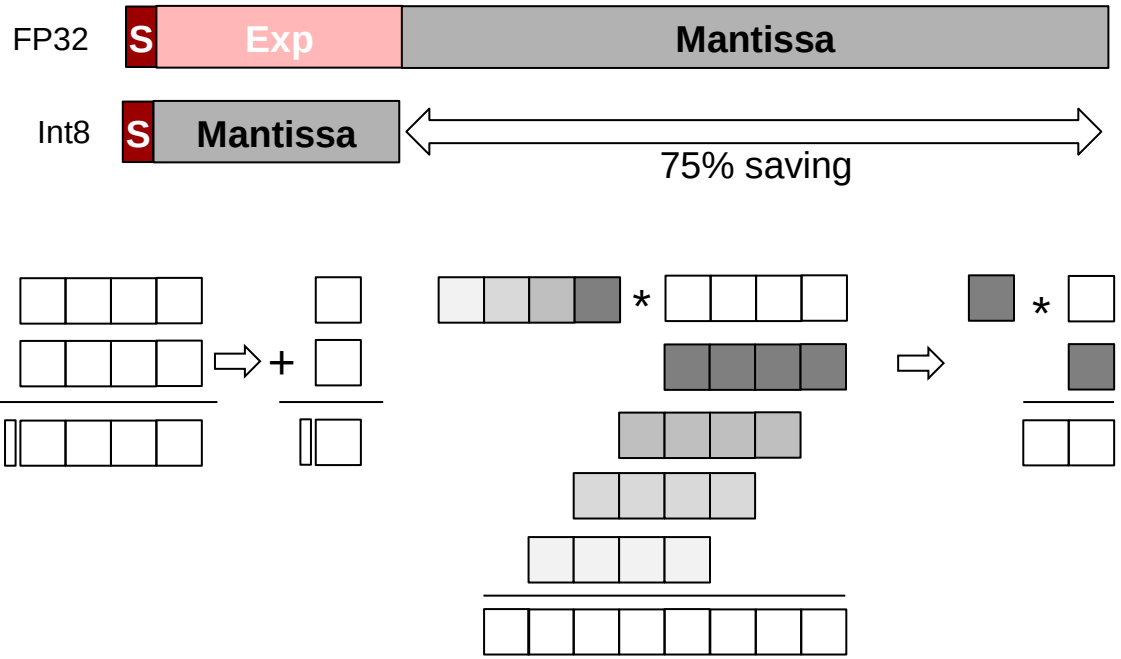
			Range	Worst-case Accuracy
FP32	1 S, 8 Exp, 23 Mantissa	Single-precision Floating-Point	3×10^{-38} ... 3×10^{38}	0.000006%
FP16	1 S, 5 Exp, 10 Mantissa	Half-precision Floating-Point	6×10^{-5} ... 6×10^{5}	0.05%
Bfloat16	1 S, 8 Exp, 7 Mantissa	Brain Float	3×10^{-38} ... 3×10^{38}	0.4%
TF32	1 S, 8 Exp, 10 Mantissa	Tensor Float	3×10^{-38} ... 3×10^{38}	0.05%
Int32	1 S, 31 Mantissa	Signed 32-bit Integer	0 ... 2×10^{9}	33%
Int16	1 S, 15 Mantissa	Signed 16-bit Integer	0 ... 3×10^{4}	33%
Int8	1 S, 7 Mantissa	Signed 8-bit Integer	0 ... 127	33%
Int4	1 S, 3 M	Signed 4-bit Integer	0 ... 7	33%

$$\max rel\ error_{INT} = \frac{|Round(1.5-\varepsilon)-(1.5-\varepsilon)|}{1.5-\varepsilon} = \frac{|1-1.5+\varepsilon|}{1.5-\varepsilon} = \frac{0.5+\varepsilon}{1.5-\varepsilon} \rightarrow \frac{1}{3}$$

$$\max rel\ error_{FP} = \frac{|Round(1+2^{-M-1}-\varepsilon)-(1+2^{-M-1}-\varepsilon)|}{(1+2^{-M-1}-\varepsilon)} = \frac{|1+2^{-M}-1-2^{-M-1}+\varepsilon|}{(1+2^{-M-1}-\varepsilon)} = \frac{2^{-M-1}+\varepsilon}{1+2^{-M-1}+\varepsilon} \approx 2^{-M-1}$$

Why Reduced-Precision Operations?

- **Space savings**
  - Memory (weights + activations/feature maps)
  - Occupies fewer bits on interconnect (memory-compute, compute-compute)
- **Area savings due to simpler arithmetic**
  - Addition of two n-bit integers: O(n)
  - Multiplication of two n-bit integers: $O(n^2)$ (*)
- **Energy savings**
  - Fewer bits need to be moved
  - Energy savings for a given pJ/Bit



	2010 (40nm)	2017 (10nm)
FP64 Fused Multiply-Add	50 pJ	8.7 pJ
64-bit read from 8kB SRAM	14 pJ	2.4 pJ
Wire energy per transition	240 fJ/bit/mm	150 fJ/bit/mm
Wire energy (256 bit, 10 mm, transition prob. 0.5)	310 pJ	150 pJ

6x 17x

Table from [1]

*) School-book style multiplication

[1] Keckler, Dally, et al. GPUs and the Future of Parallel Computing. In IEEE Micro, 31(5), 2010.

Reduced Precision? Where?

- **Weights**
  - Fit more weights on the device (phone, micro-controller)
- **Activations**
  - Feature Maps in CNNs often too large to fit into on-chip memory
  - Example:

    ResNet-50 on [256, 3, 224, 224] input (ImageNet with batch size 256, NCHW)

    Largest Conv2d feature map: [256, 256, 56, 56] x FP32 → 784 MiB!
- **Gradients**
  - Reduce communication cost in **`all-reduce`** operation of data parallel training.
  - 1-bit SGD [2] inspired by $\Sigma\Delta$-modulation, QSDG [3] randomized rounding + Elias coding, TernGrad [4] generates gradient vector with elements $\in\{-1, 0, +1\}$

[2] Seide et al. 1-Bit Stochastic Gradient Descent and its Application to Data-Parallel Training of Speech DNNs. In Interspeech'14.
[3] Alistarh et al. QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding. In NIPS'17.
[4] Wen et al. TernGrad: Ternary Gradients to Reduce Communication in Distributed Deep Learning. In NIPS'17.

Reduced Precision? How?

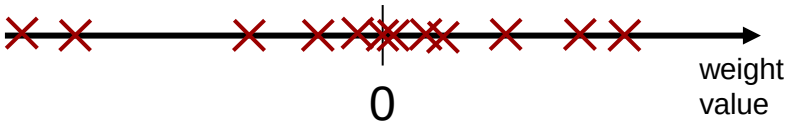
- **FP32 → FP16/Bfloat16/TF32**
  - **Post-training**: just convert trained model (e.g., via TensorFlow Lite `TFLiteConverter`), that usually works directly
  - Training in TF16 or **mix-precision training**: use Keras mixed-precision API, but pay attention to gradients (cancellations in Bfloat16, overflows (inf) in FP16)
  - Availability of **16-bit floating-point allows on-device training** (re-training)

- **FP32 → Integer: Quantization**
  - Post-training: Quantization via Calibration Dataset
  - Re-training with Quantization
  - Both can be done using the Ascend Model Compression Toolkit (AMCT) [5]

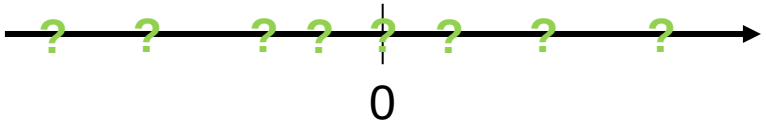
[5] CANN V199R929C10 Development Auxiliary Tool Guide. Huawei.
https://support.huawei.com/enterprise/en/doc/EDOC1100164848/aa7900c5/ascend-model-compression-toolkit-instructions-tensorflow

Weight Quantization

Weights in NNs are centered around zero

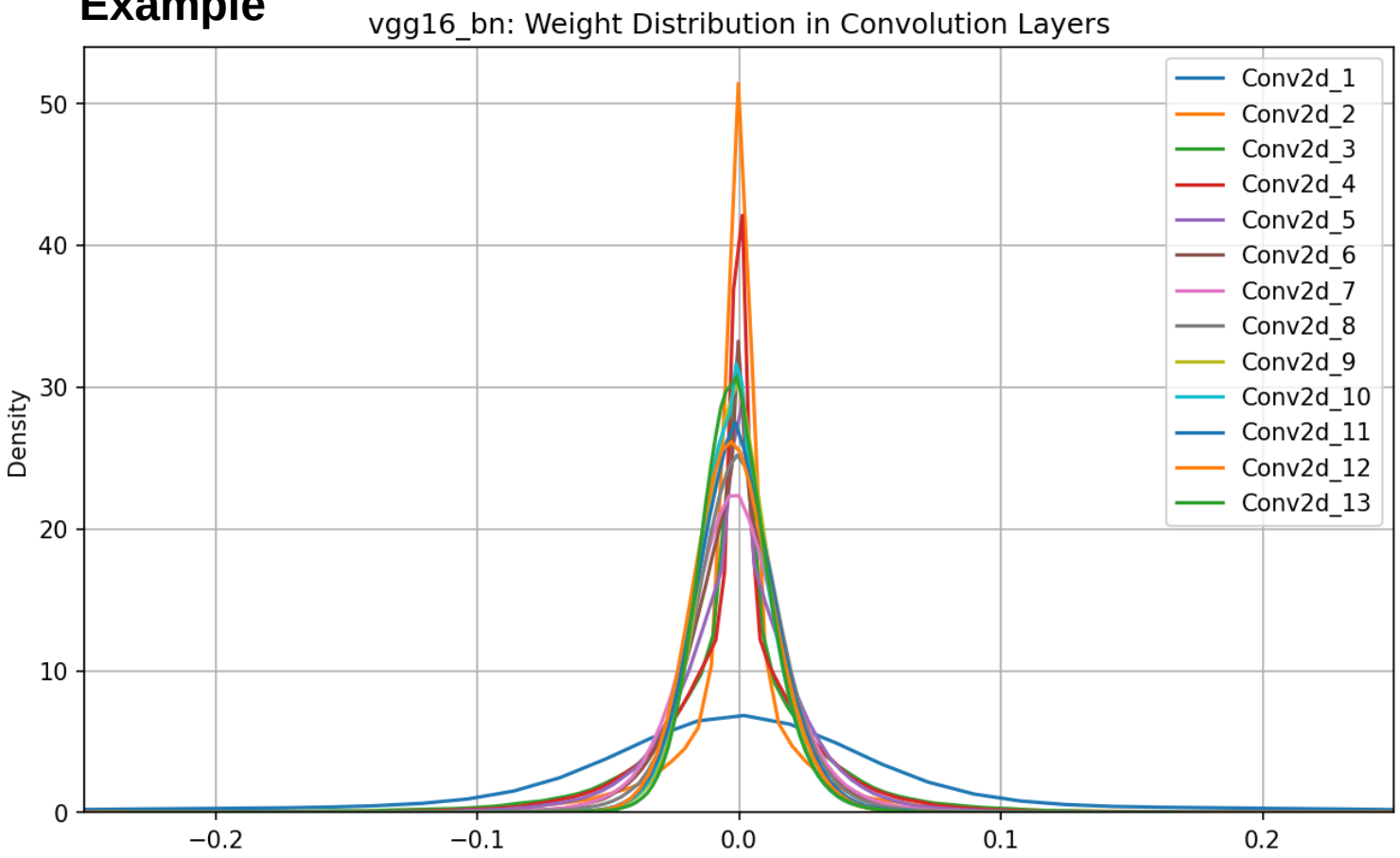


Quantization using only a subset



n-bit quantization → 2^n points

Example



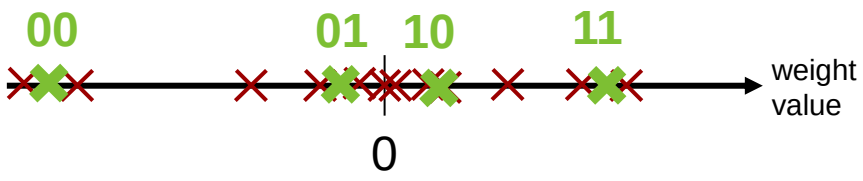
Quantiziation: Codebook Approach

= weight sharing (weights in same cluster share value)

- For a given FP32 weight matrix $\boldsymbol{W} \in \mathbb{R}^{n \times m}$ perform **k-means clustering**, with $l = 2^b$ centroids, chosen that

$$\sum_{i=1}^{m} \sum_{j=1}^{n} \sum_{k=1}^{l} \left\| w_{ij} - c_k \right\|_2^2 \longrightarrow \min \quad .$$

  - Select the closest centroid $c_{k'}$ when encoding w_{ij}. → Encode k' using b bits. [6]

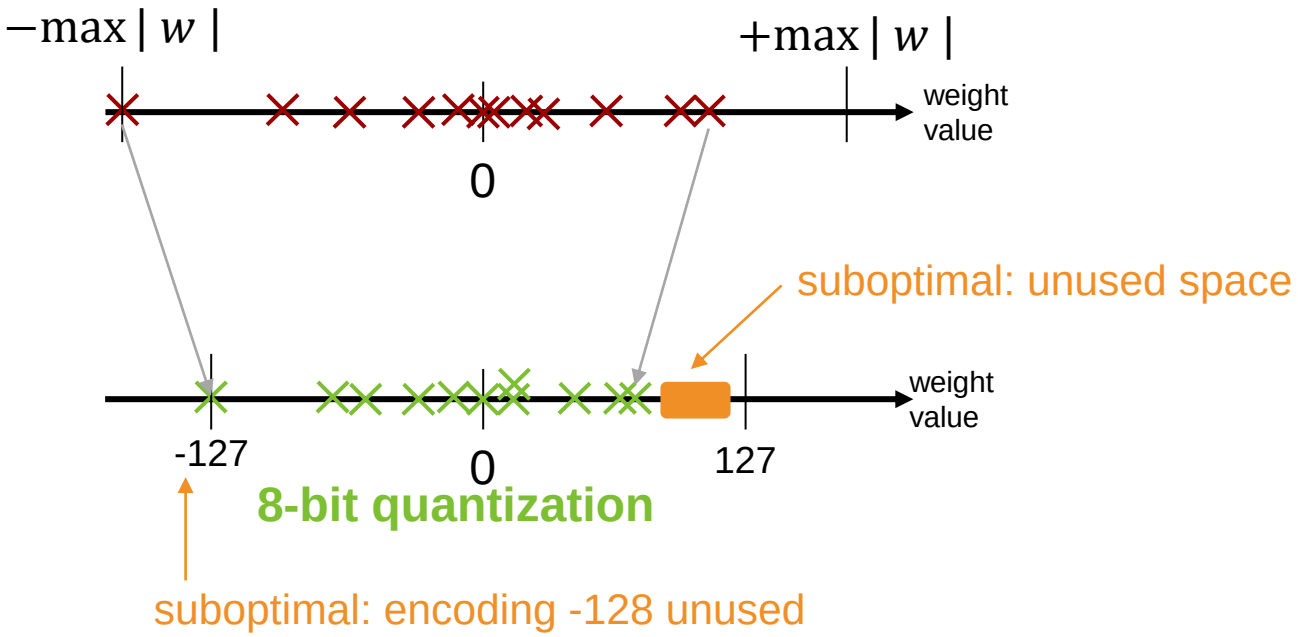


  - Store centroids $c_1, \ldots, c_{2^b}$ in codebook of length $2^b \times$ FP32.
  - Decode k': Lookup $c_{k'}$ in codebook.
  - Han et al. [7] proposes retraining the network to fine-tune the centroids. Backpropagation updates centroids (w/o reassigning weights to cluster centroids)

[6] Gong et al. Compressing Deep updates Networks using Vector Quantization. arXiv 1412.6115. 2014.
[7] Han et al. Deep Compression: Compressing DNNs with Pruning, Trained Quantization, and Huffman Coding. ICLR 2015.

Linear Quantization

- **Symmetric Quantization** $Q(w) = \text{Round}\left(\frac{2^{b-1}-1}{\max|w|}w\right)$



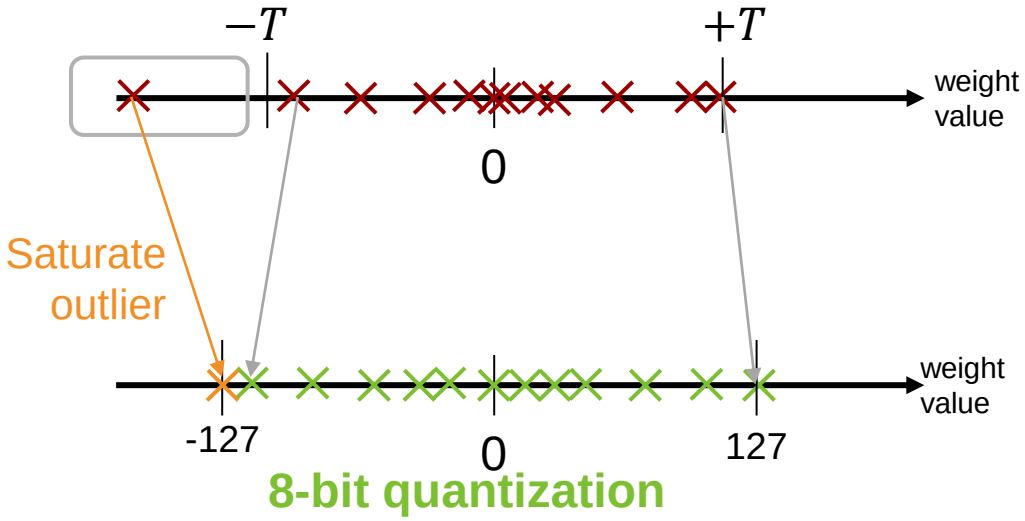
Encoding is inefficient:
Outliers increase distance between codes
→ unused space but large accuracy loss

max(..) aggregate function is very sensitive to outliers.

Robust Linear Quantization

- **Saturate values $|w| > T$**

$$Q(w) = \max\left[\min\left(\text{Round}\left(\frac{2^{b-1}-1}{T}w\right), 2^{b-1}-1\right), -2^{b-1}+1\right]$$



How do we choose Threshold T?

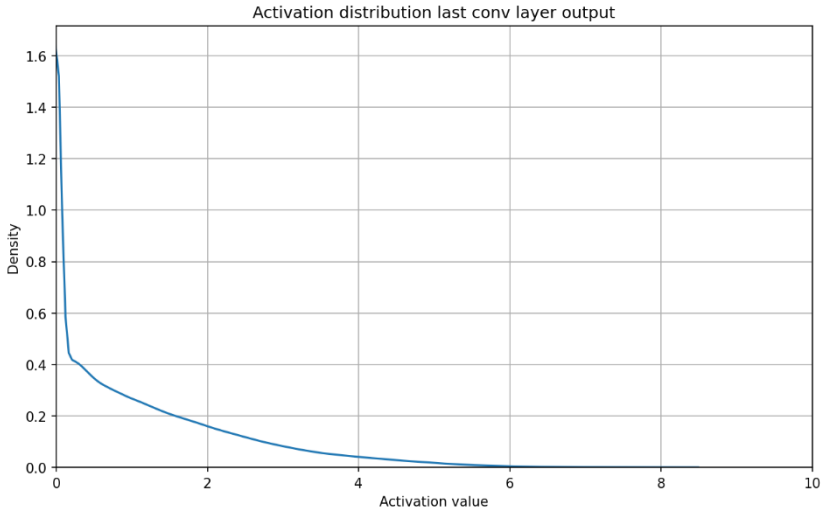
Quantization: Information Theory Perspective

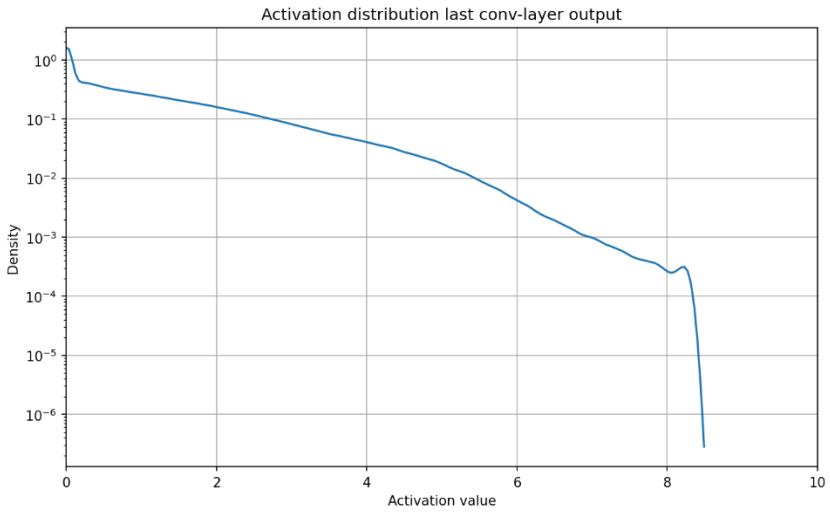
- Quantized **`INTx`** model should contain the **same information** as the FP32 model.
- Find quantization that **minimizes the information loss**.
- **Approach**: treat trained model applied on a given dataset as from some distribution $P(x)$, quantized values from a different distribution $Q(x)$.

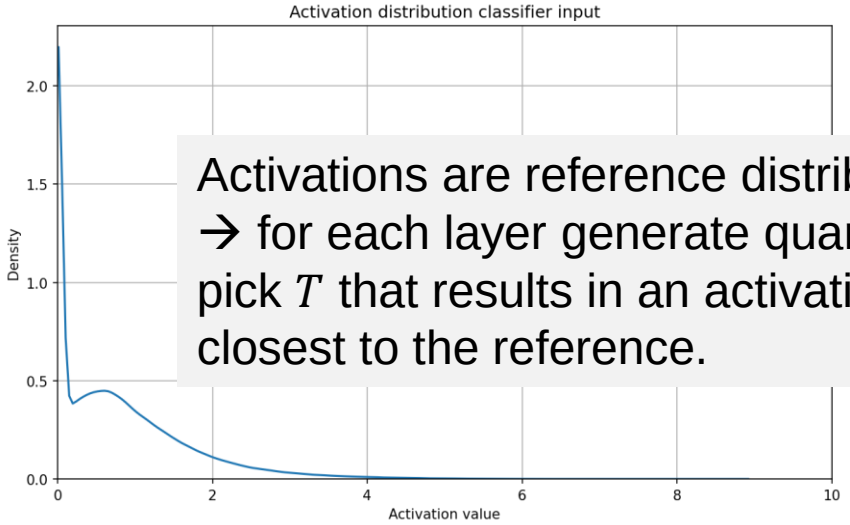
- Use Entropy of empirical distribution as a measure of information.
- Find quantization $Q(x)$ that **minimizes relative entropy** (= information loss)
- Relative entropy is the **Kullback-Leibler (KL) Divergence** between $P(x)$ and $Q(x)$:
  - $D_{KL} = -\sum_x P(x) \log\left(\frac{P(x)}{Q(x)}\right) \longrightarrow \min$.
- Pick Threshold T that KL Divergence is minimized by **iteratively** sweeping T while gathering statistics of activations on **calibration dataset** for each layer. [8]

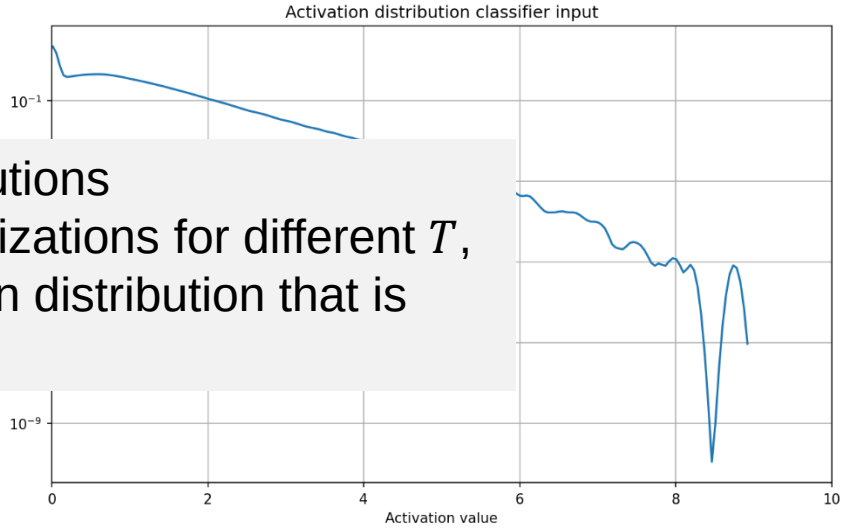
[8] Migacz. 8-bit Inference with TensorRT. In GTC, 2017.

Example: Distribution of Activations

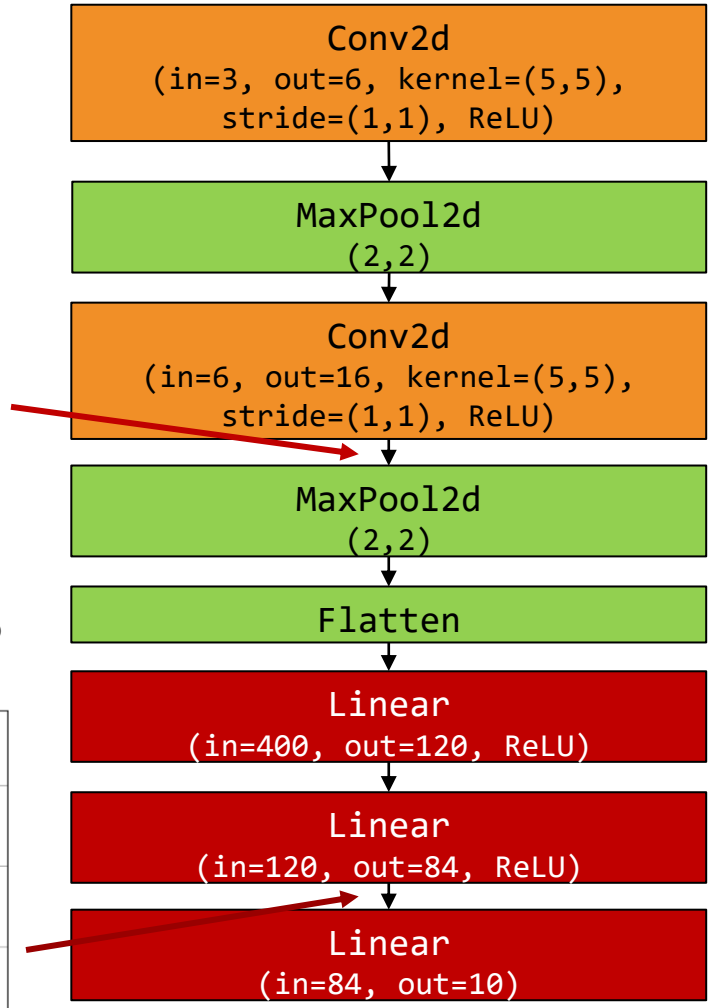




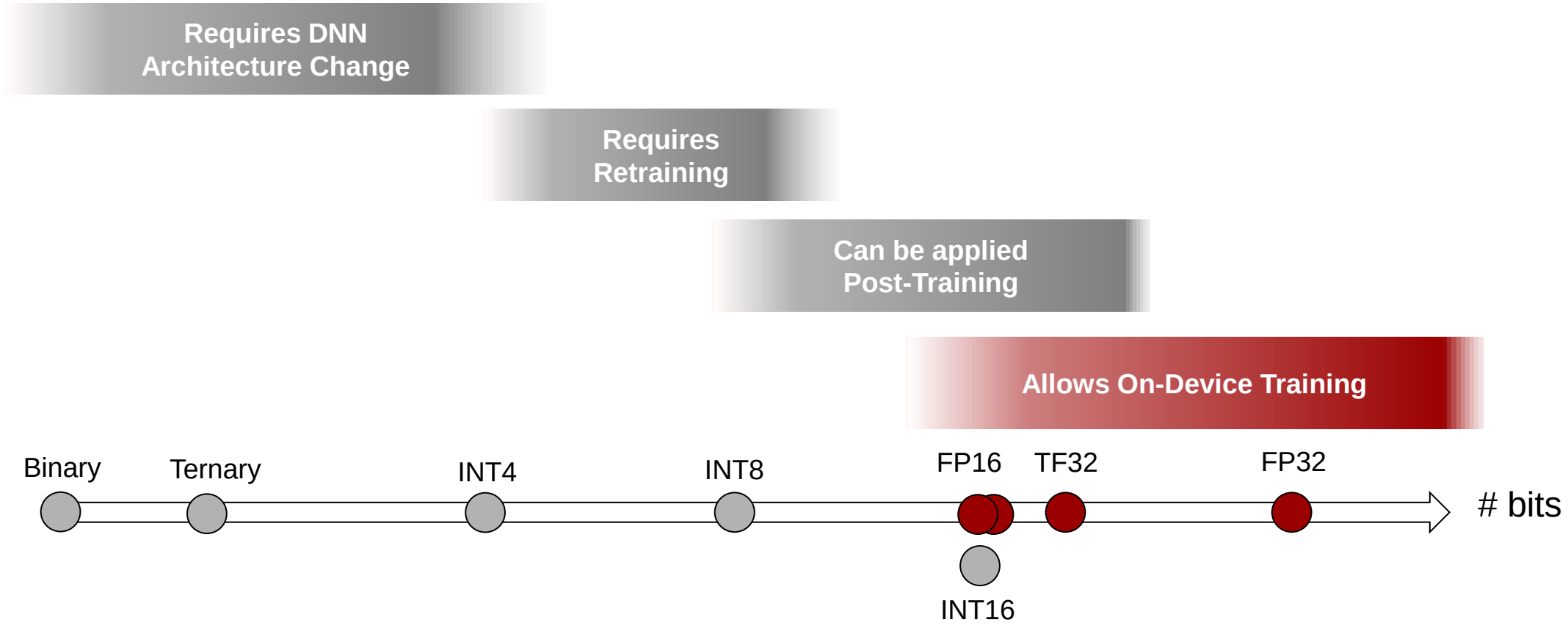




Activations are reference distributions
→ for each layer generate quantizations for different T, pick T that results in an activation distribution that is closest to the reference.



How-low can one go?



Binary Neural Networks and XNOR Networks

weights in convolution kernel (filter)

- **Binary Weight Networks: Weights $\boldsymbol{w} \in \{-1, +1\}$, real-value tensors**
  - Convolution without multiplication: w =+1 → addition, $w = -1$ → subtraction
- **XNOR-Networks: Binary Weights and Tensors**
  - Mapping Binary value -1 → **0**, value +1 → **1**
  - Multiplication: XNOR (element-wise multiplication of vector → bitwise XNOR)

x_i	y_i	$x_i * y_i$
-1	-1	+1
-1	+1	-1
+1	-1	-1
+1	+1	+1

⟺

a_i	b_i	a_i XNOR b_i
0	0	1
0	1	0
1	0	0
1	1	1

(popcount: number of 1s in bitvector)

  - Dot-product $\langle \boldsymbol{x}, \boldsymbol{y} \rangle$ of vectors $\boldsymbol{x}, \boldsymbol{y}$ $\Leftrightarrow \mathrm{popcount}(\boldsymbol{a} \text{ xnor } \boldsymbol{b})$

[9] Rastegari et al. XNOR-Net: ImageNet Classification Using Binary Convolution Neural Networks. In ECCV 2016.
[10] Rastegari et al. Enabling AI at the Edge with XNOR-Networks. In Comm of the ACM. Vol 63, No. 12, pp 83-90, 2020.

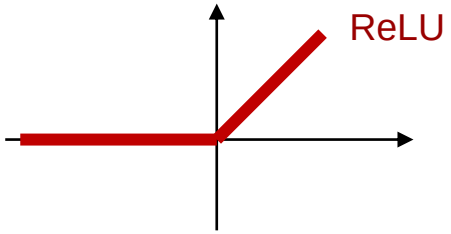
Binary Neural Networks and XNOR Networks

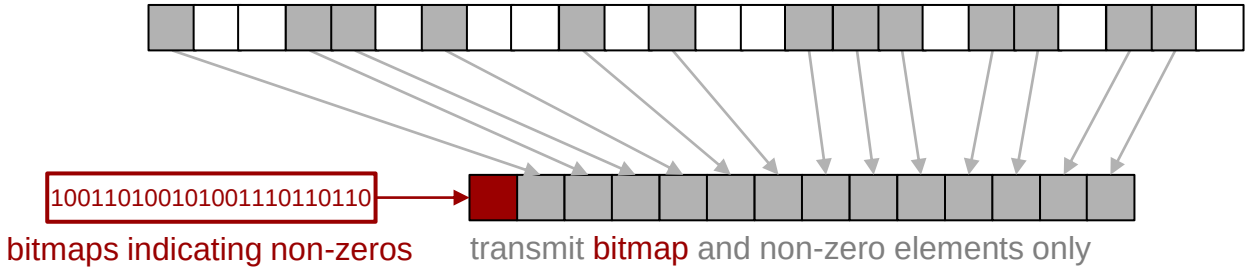
- Training is challenging, binarization (`sign` is non differentiable everywhere and where it is derivative is zero) use $\max(-1, \min(1, x))$ or identity function for backprop
- Sometimes a distillation approach can be used with a teacher in full-precision and binary student network.
- Batch Norm cannot be fused into weights (as a linear transformation as you can in higher precision)
- Binary neural networks almost always require architecture changes to obtain similar performance as full precision networks, e.g., adding more layers.
  - → diminishing returns when accuracy is pushed to high

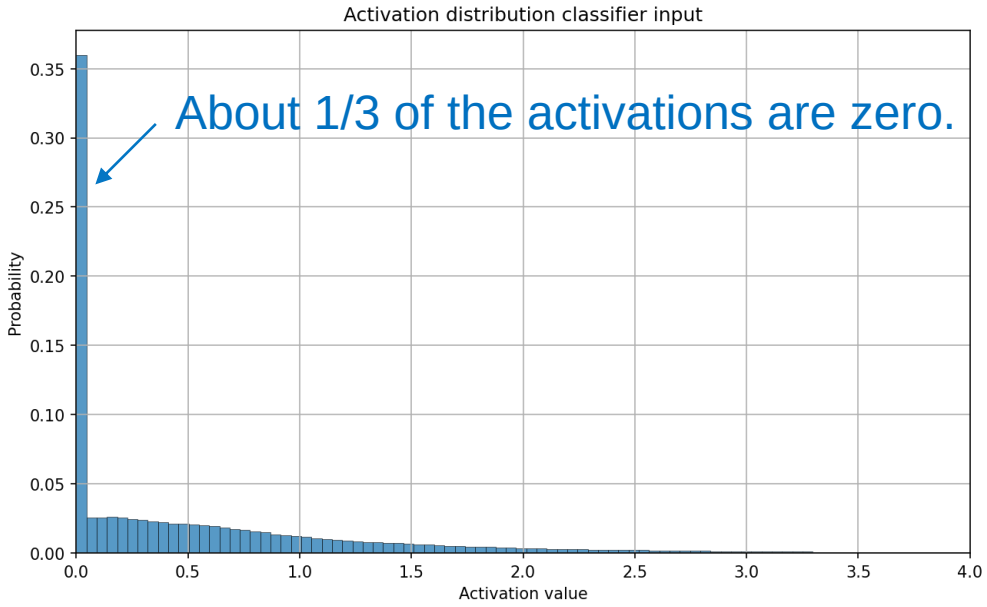
Compression: Sparsity of Activations

- **ReLU is extremely popular activation function**
  - Non-vanishing gradient > 0
- **Output of ReLU is 0 for for ½ of the inputs → sparse output**
- **Do you need to process zeros?**
  - Zero-bit use zero-bit line to “disable” multiplier → saves energy.
- **Do you need to transfer zeros?**

  → Zero-value Compression [12]



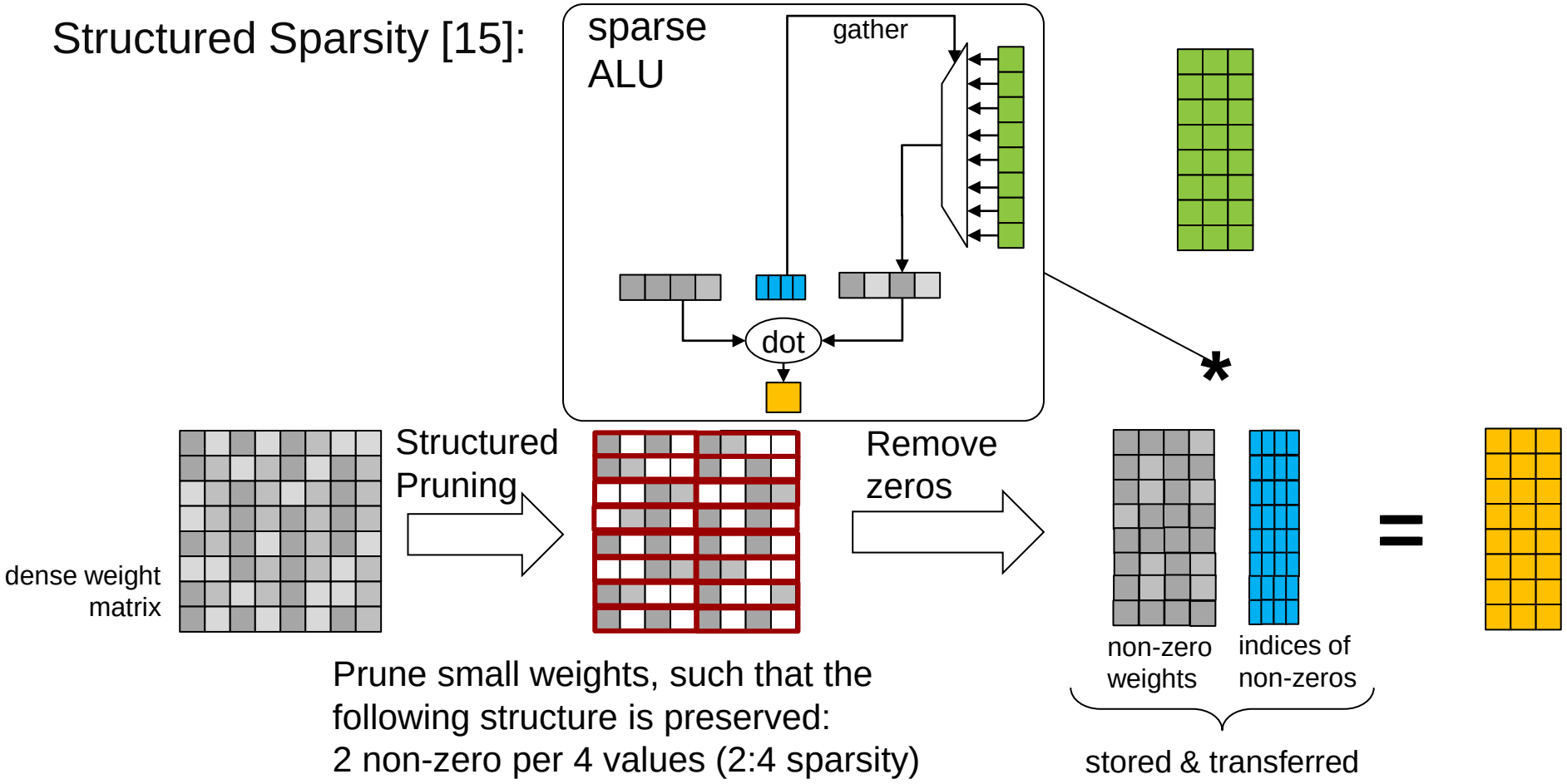




[12] Rhu et al. Compressing DMA Engine: Leveraging Activation Sparsity for Training Deep Neural Networks. In HPCA 2018.

Compression: Sparsity of Models

- Weight pruning for regularization (at the cost of decreased noise resiliency [14]).
- Structured Sparsity [15]:



[14] Liebenwein: The Effect of Pruning Neural Networks beyond Test Accuracy. In MLSys 2021.
[15] NVIDIA. A100 Whitepaper. 2020.

HUAWEI

Outline

1) Quantization

2) **Huawei DaVinici Core and Ascend SoC**

Deep Learning Hardware

Huawei DaVinci Core and Ascend SoC

based on Presentation by Heiko Joerg Schick (Chief Architect Advanced Computing, HiSilicon)

https://www.schihei.eu/blog/archives/03-2021

Technology Scalable Across Device Classes

	Device				Edge		Cloud
	Earphone	**Always-on**	**Smartphone**	**Laptop**	**IPC**	**Edge Server**	**Data Centre**
Compute	20 MOPS	100 GOPS	1 - 10 TOPS	10 - 20 TOPS	10 - 20 TOPS	10 - 100 TOPS	200+ TOPS
Power budget	1 mW	10 mW	1 - 2 W	3 - 10 W	3 - 10 W	10 - 100 W	200+ W
Model size	10 KB	100 KB	10 MB	10 - 100 MB	10 -100 MB	100+ MB	300+ MB
Latency	< 10 ms	~10 ms	10 - 100 ms	10 - 500 ms	10 - 500 ms	ms ~ s	ms ~ s
Inference?	Y	Y	Y	Y	Y	Y	Y
Training?	N	N	Y	Y	Y	Y	Y
SoC scale	Nano	Tiny	Lite	Mini Ascend 310	Mini Ascend 310	Multi-Mini Ascend 310	Max Ascend 910

Two System-on-Chip (SoC)

Inference

Ascend 310

AI SoC with ultimate efficiency

- Half precision (FP16): 8 TFLOPS
- Integer precision (INT8): 16 TOPS
- 16-channel full-HD video decoder: H.264/265
- 1-channel full-HD video encoder: H.264/265
- Max. power consumption: 8 W
- 12nm

Training

Ascend 910

Highest compute density on a single chip

Atlas Products with Ascend Accelerators

52 mm x 38 mm x 10 mm

Atlas 300

AI Accelerator Card

- 64 TOPS of INT8 @ 67 W
- 32 GB memory
- 64-channel HD video real-time analytics
- Standard half-height half-length PCIe card form factor, applicable to general-purpose servers

reddot award 2019 winner

Atlas 200

AI Accelerator Module

- 16 TOPS of INT8
- 16-channel HD video real-time analytics, JPEG decoding
- 4 GB/8 GB memory, PCIe 3.0 x4 interface
- Operating temperature: -25°C to +80°C

Atlas 500

AI Edge Stations

- 16 TOPS of INT8
- 25–40 W
- Wi-Fi & LTE
- 16-channel HD video real-time analytics
- Fanless design, -40°C to +70°C environments

reddot award 2019 winner

Atlas 800

Deep Learning System

- Plug-and-play installation
- Ultimate Performance
- Integrated Management

Ascend 310

AI SoC with ultimate efficiency

- Half precision (FP16): 8 TFLOPS
- Integer precision (INT8): 16 TOPS
- 16-channel full-HD video decoder: H.264/265
- 1-channel full-HD video encoder: H.264/265
- Max. power consumption: 8 W
- 12nm

Ascend 910

Highest compute density on a single chip

- Half precision (FP16): 256 TFLOPS
- Integer precision (INT8): 512 TOPS
- 128-channel full-HD video decoder: H.264/265
- Max. power consumption: 350 W
- 7nm

Atlas 900 AI Cluster

The pinnacle of computing power

- Thousands of Ascend 910 AI processors
- High-speed interconnection
- Delivers up to 256 to 1024 PetaFLOPS at FP16
- Can complete model training based on ResNet-50 within 59.8 seconds
- 15% faster than the second-ranking product
- Faster AI model training with images and speech

Atlas 200 DK

Quickly build development environments in 30 minutes

- 16 TOPS of INT8 @ 24 W
- 1 USB type-C, 2 camera interfaces, 1 GE port, 1 SD card slot
- 4 GB/8 GB memory

Kunpeng 920

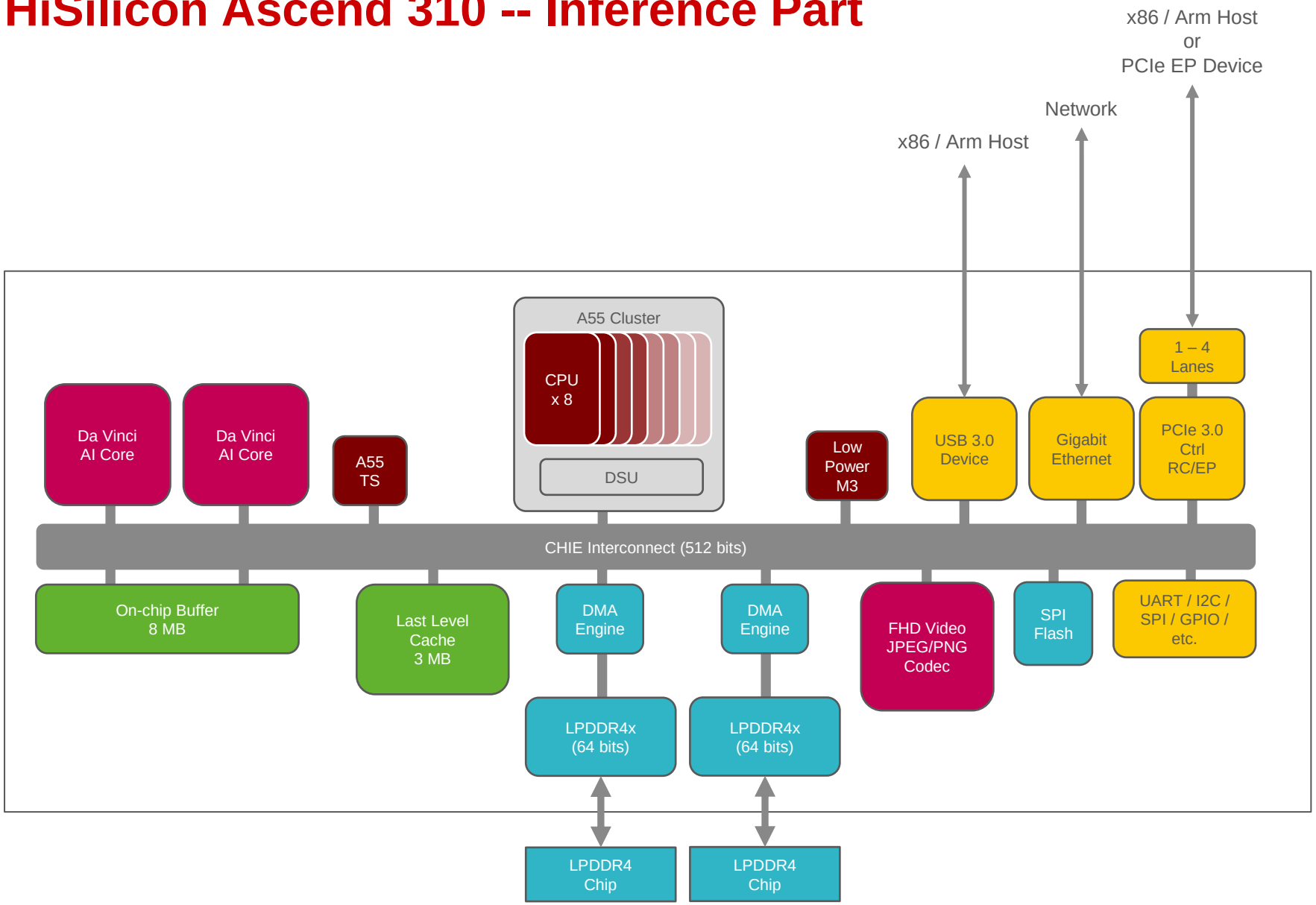
The industry's highest-performance ARM-based server CPU

- ARM v8.2-architecture
- up to 64 cores, 2.6 GHz
- 8 DDR4 memory channels
- PCIe 4.0 and CCIX
- Integrated 100GE LOM and encryption and compression engines
- Supports 2- or 4-socket interconnects

Storage-intensive — **Computing-intensive**

5290	5280	2280	1280	2480	X6000
4U 72-drive storage model	4U 40-drive storage model	2U 2S balanced model	1U 2S high-density model	2U 4S high-performance model	2U 4-node high-density model

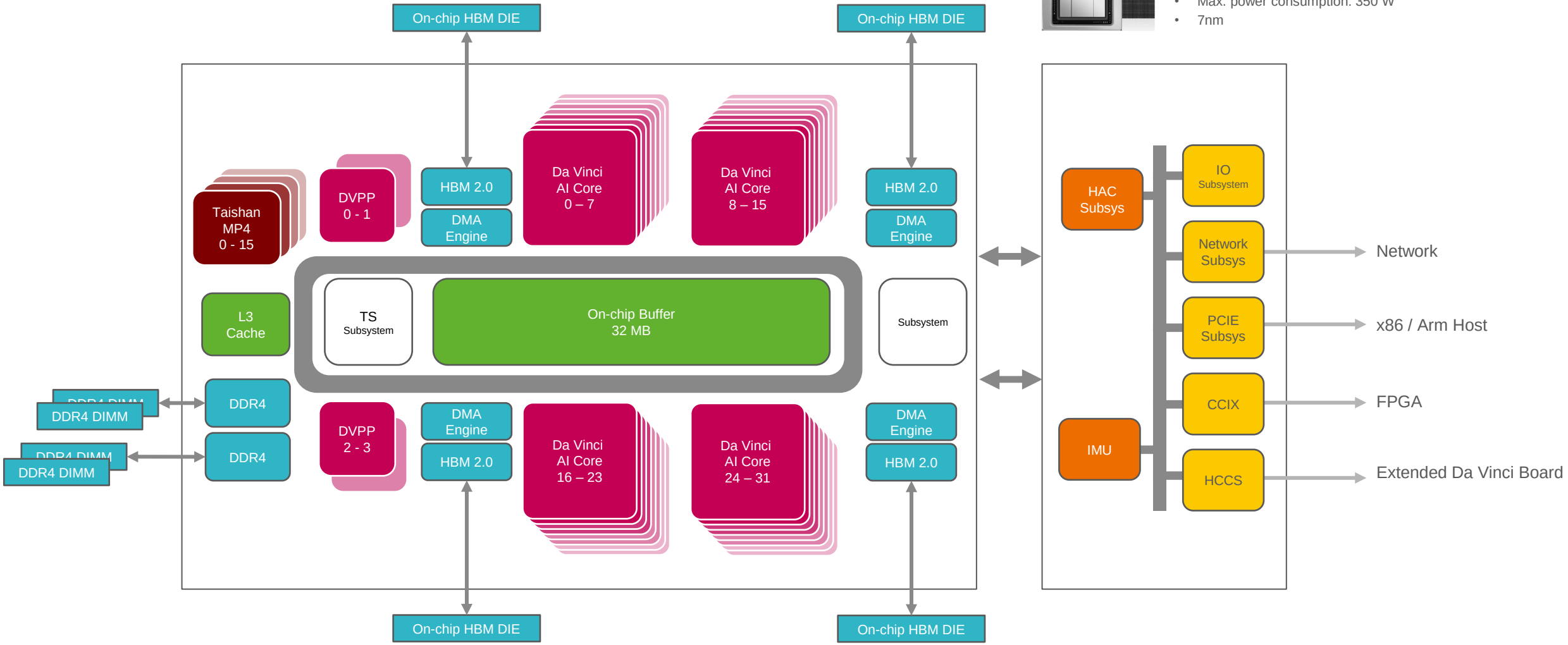
HiSilicon Ascend 310 -- Inference Part



- Half precision (FP16): 8 TFLOPS
- Integer precision (INT8): 16 TOPS
- 16-channel full-HD video decoder: H.264/265
- 1-channel full-HD video encoder: H.264/265
- Max. power consumption: 8 W
- 12nm

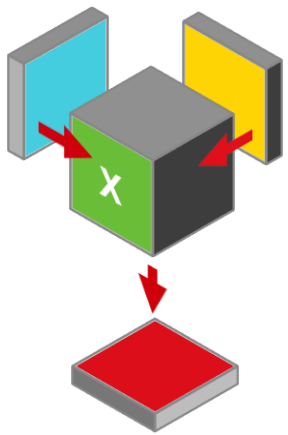
HiSilicon Ascend 910 -- Training Part

- Half precision (FP16): 256 TFLOPS
- Integer precision (INT8): 512 TOPS
- 128-channel full-HD video decoder: H.264/265
- Max. power consumption: 350 W
- 7nm

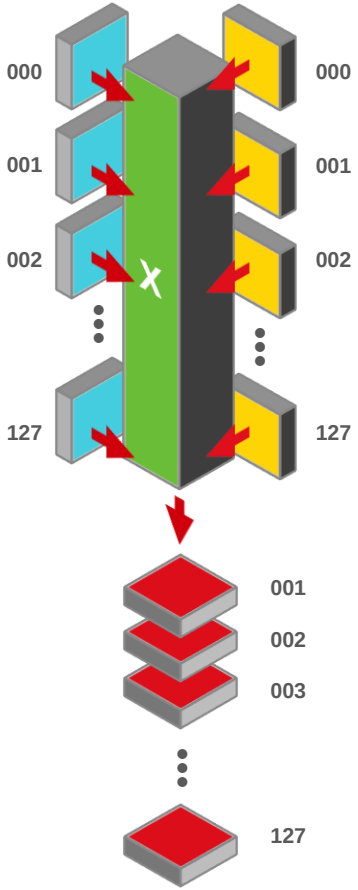


Building Blocks for Deep Learning Accelerators

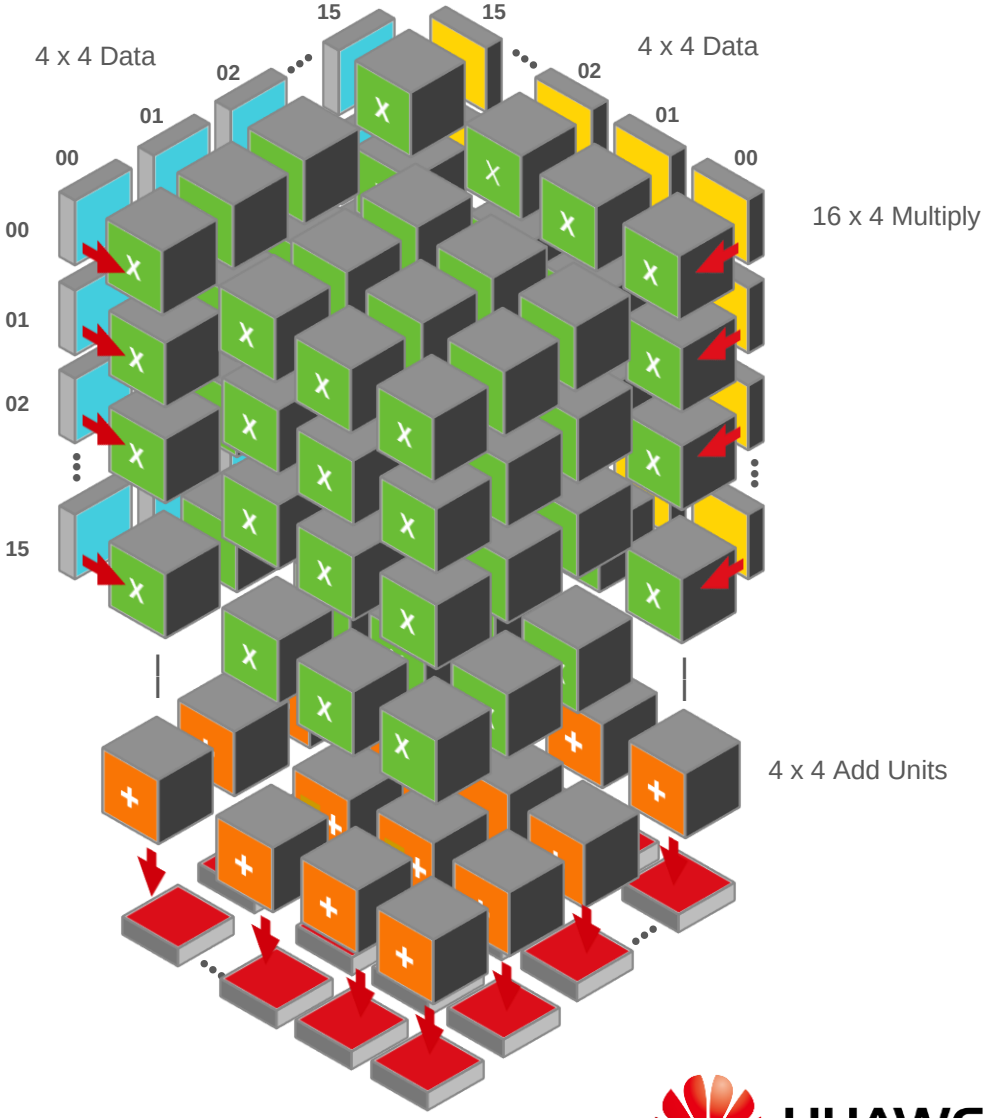
Scalar Unit
Full flexibility in computation



Vector Unit
Rich and efficient operations



Cube Unit
High intensity computation



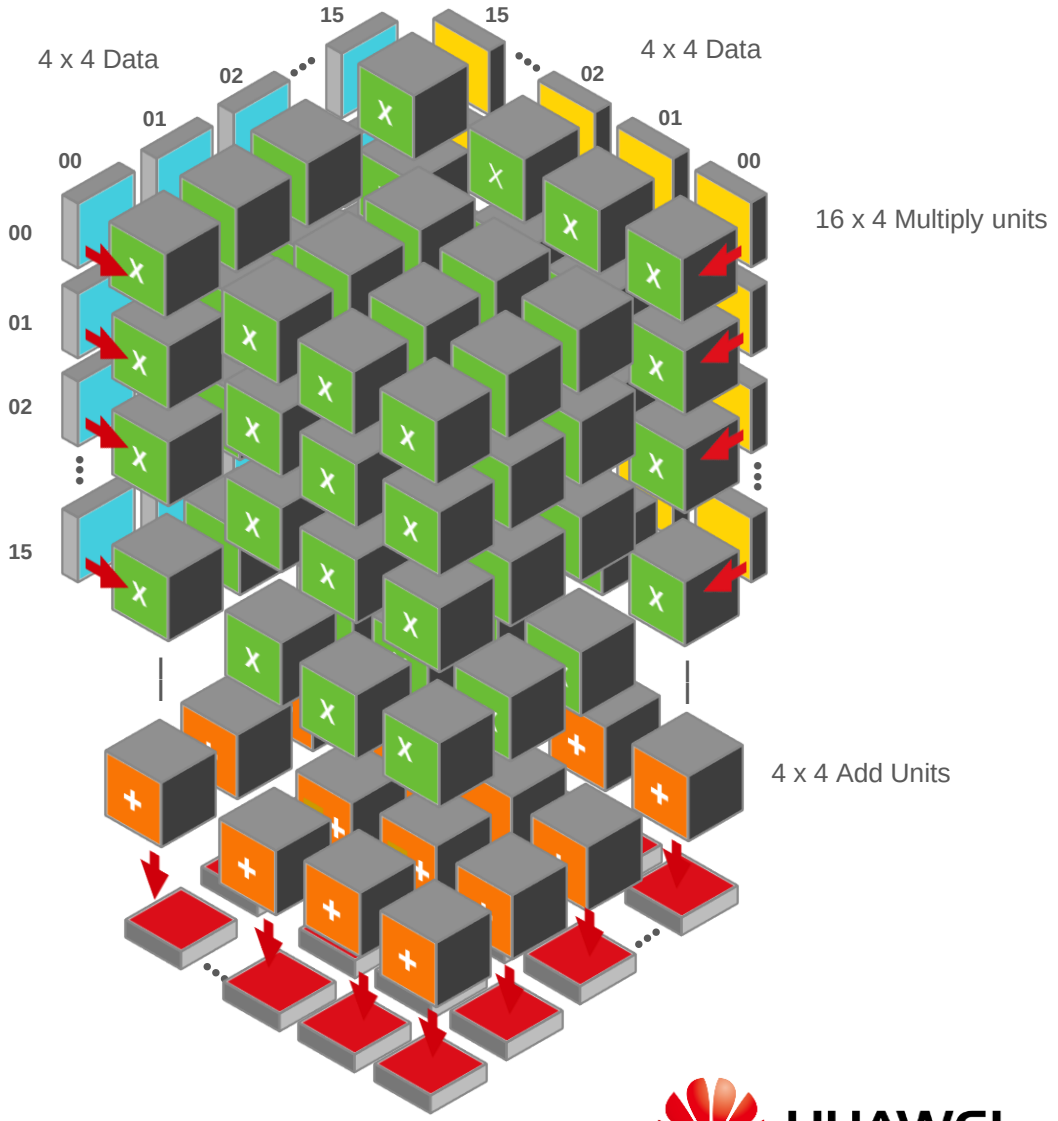
HUAWEI

Building Blocks for Deep Learning Accelerators

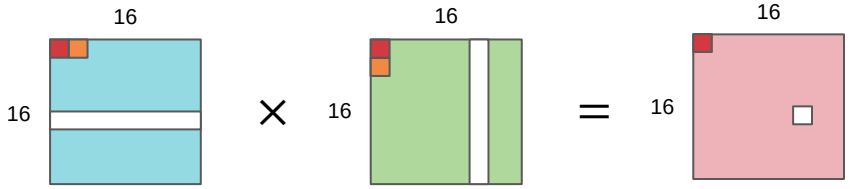
N	N^2	N^3
1	1	1
2	4	8
4	16	128
8	64	512
16	**256**	**4,096**
32	1,024	32,768
64	4,096	262,144

	GPU + Tensor core	AI core + SRAM
Area (normalized to 12 nm)	5.2 mm^2	13.2 mm^2
Compute power	1.7 TFLOPS (FP16)	8 TFLOPS (FP16)

Cube Unit
High intensity computation



Advantages of Special Compute Units




```
float a[16][16], b[16][16], c[16][16];
```

Scalar:

```
for (int i=0; i<16; i++) {
    for (int j=0; j<16; j++) {
        for (int k=0; k<16; k++) {
            c[i][j] += a[i][k] * b[k][j];
        }
    }
}
```

$Cycles = 16 * 16 * 16 * 2 = 8192$
$Data\ per\ cycle = Rd\ 3;\ \ Wr\ 1$

Vector:

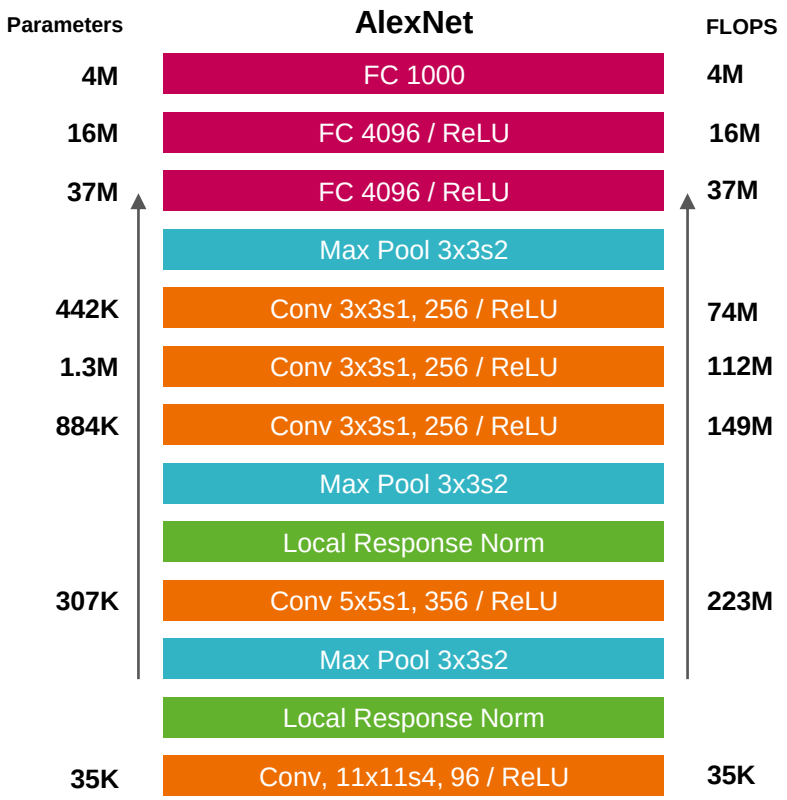
```
for (int i=0; i<16; i++) {
    for (int j=0; j<16; j++) {
        c[i][j] = a[i][:] *+ b[:][j];
    }
}
```

$Cycles = 16 * 16 = 256$
$Data\ per\ cycle = Rd\ 2 * 16;\ \ Wr\ 16$

Cube:

```
c[:][:] = a[:][:] X b[:][:]
```

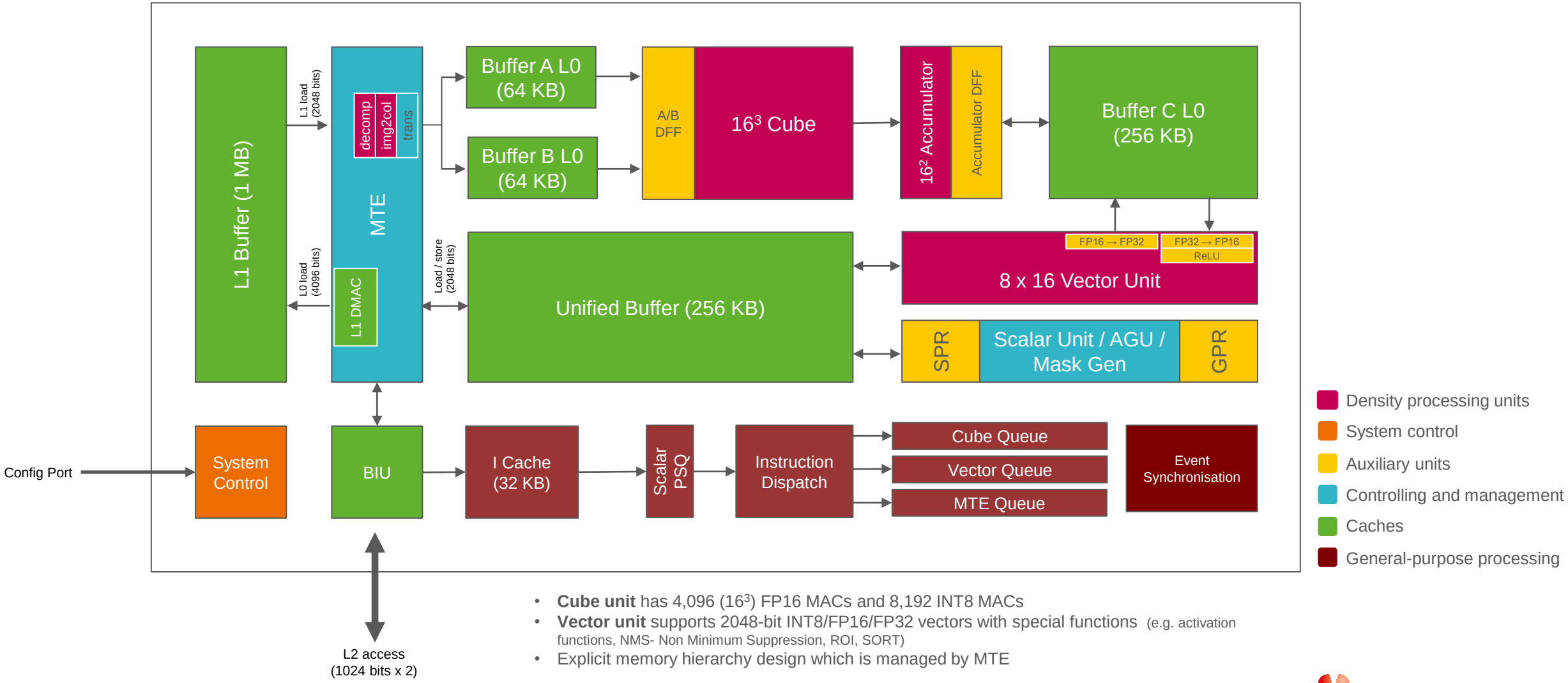
$Cycles = 1$
$Data\ per\ cycle = Rd\ 2 * 16 * 16;\ \ Wr\ 16*16$



Number of parameters and floating point operations per second (FLOPS) for each layer of the AlexNet artificial intelligence model.

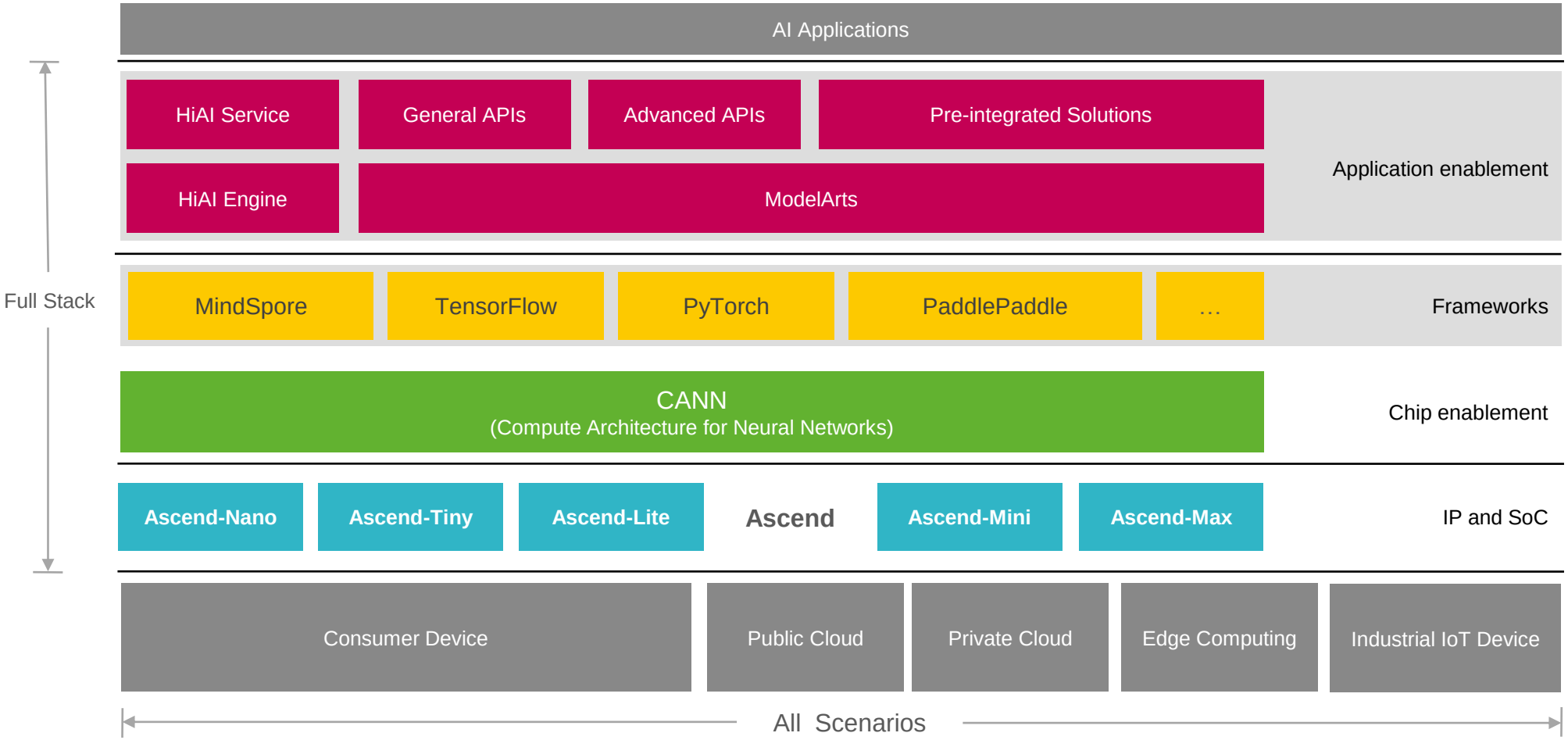
99% of the computations are matrix-matrix multiplications	Typical CNN networks		
	AlexNet	**VGG16**	**Inception-v3**
Model memory (MB)	> 200	> 500	90-100
Parameter count (Million)	60	138	23.2
Computation amount (Million)	720	15300	5000

DaVinici Core Architecture



- **Cube unit** has 4,096 (16^3) FP16 MACs and 8,192 INT8 MACs
- **Vector unit** supports 2048-bit INT8/FP16/FP32 vectors with special functions (e.g. activation functions, NMS- Non Minimum Suppression, ROI, SORT)
- Explicit memory hierarchy design which is managed by MTE

Ascend AI Software Stack



Application enablement:
Full-pipeline services (ModelArts), hierarchical APIs, and pre-integrated solutions

MindSpore:
Unified training and inference framework for device, edge, and cloud (both standalone and cooperative)

CANN:
SoC operator library and highly automated development toolkit

Ascend:
AI IP and SoC series based on unified scalable architecture

MindSpore Deep Learning Framework

https://www.mindspore.cn/en

MindSpore

```
import mindspore.dataset as ds
from mindspore import context, nn, ops
from mindspore.train import Model
from mindspore.train.callback import LossMonitor

class Net(nn.Cell):
    def __init__(self, input_dim, hidden1_dim, hidden2_dim):
        super().__init__()
        self.fc1 = nn.Dense(in_channels=input_dim,
            out_channels=hidden1_dim, activation='relu')
        self.fc2 = nn.Dense(in_channels=hidden1_dim,
            out_channels=hidden2_dim, activation='relu')
        self.fc3 = nn.Dense(in_channels=hidden2_dim,
            out_channels=1)

    def construct(self, input_x):
        h1 = self.fc1(input_x)
        h2 = self.fc2(h1)
        return self.fc3(h2)


class Loss(nn.Cell):
    def __init__(self):
        super().__init__()
        self.loss_fn = ops.SigmoidCrossEntropyWithLogits()
        self.mean_fn = ops.ReduceMean()

    def construct(self, logits, labels):
        return self.mean_fn(self.loss_fn(logits, labels))
```

Graph Mode: static compute graph (~ non-eager execution)

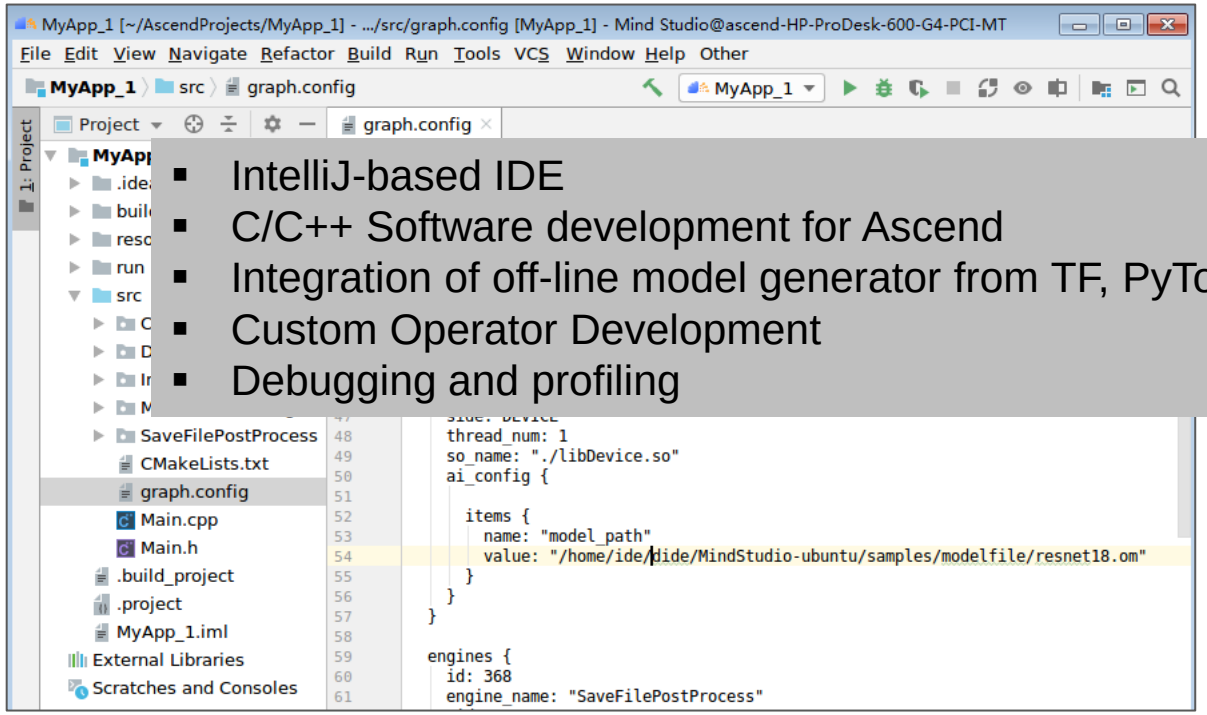
Execution on Ascend (other options ‘GPU’ or ‘CPU')

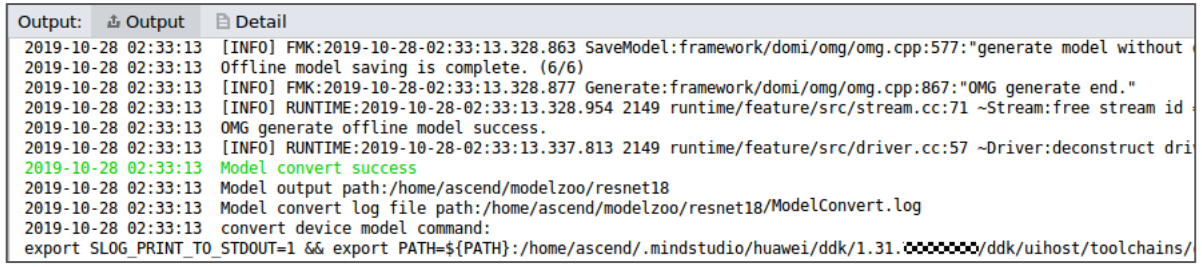
```
...
context.set_context(
    mode=context.GRAPH_MODE,
    device_target='Ascend')
dataset = create_dataset(num_rows=10000, input_dim=5)
dataset = dataset.batch(batch_size=128)

net = Net(input_dim=5, hidden1_dim=100, hidden2_dim=20)
loss_obj = Loss()
net_opt = nn.Adam(net.trainable_params())
model = Model(network=net, loss_fn=loss_obj, optimizer=net_opt)

model.train(epoch=400, train_dataset=dataset,
    callbacks=[LossMonitor(per_print_times=100)],
    dataset_sink_mode=False)
```

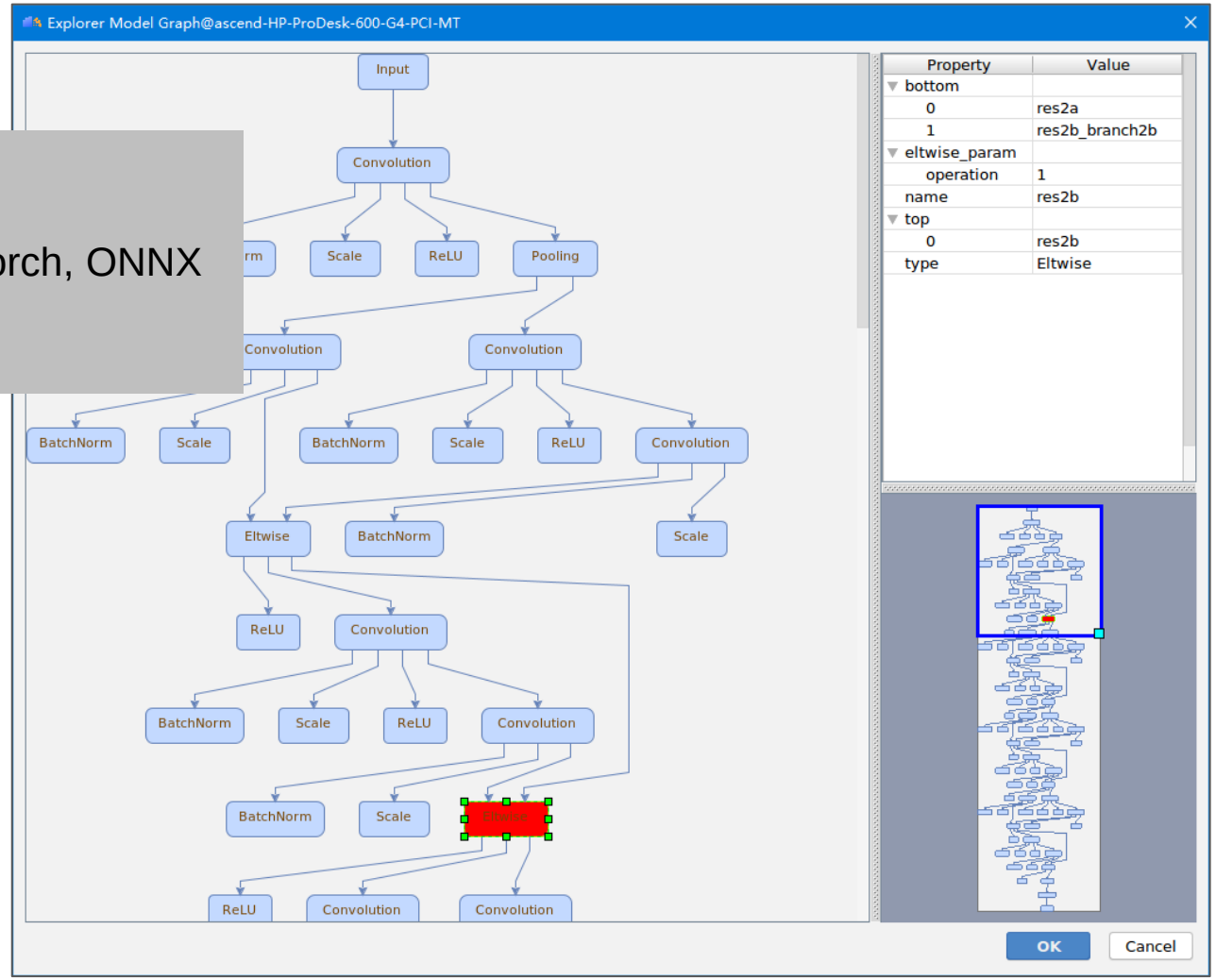
https://www.hiascend.com/software/mindstudio

MindStudio IDE

- IntelliJ-based IDE
- C/C++ Software development for Ascend
- Integration of off-line model generator from TF, PyTorch, ONNX
- Custom Operator Development
- Debugging and profiling







HUAWEI

https://developer.huawei.com/consumer/en/hiai

HiAI Engine and HiAI Foundation (HMS)

- **HMS (Huawei Mobile Services) set of services for the mobile platform**
- **AI related components:**

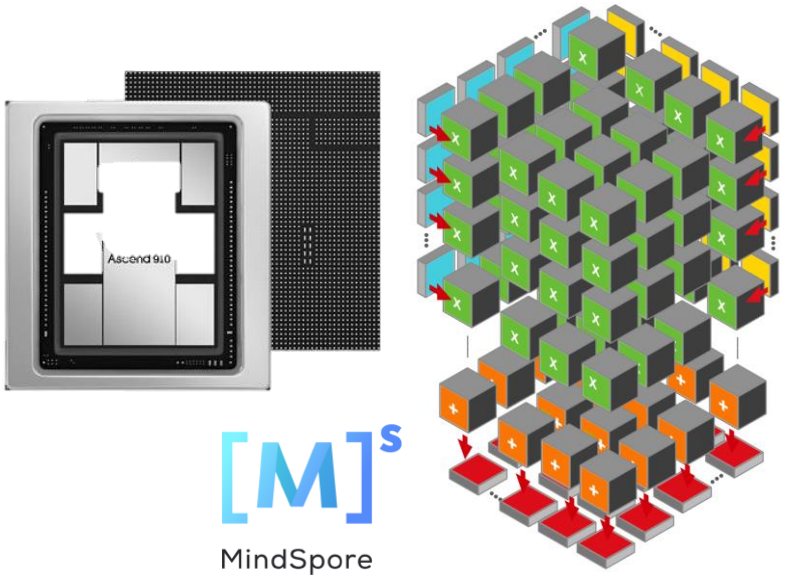
Cloud-enabled Services: Abilities in the cloud
AI Voice, AI Lens, and distribution.

Use existing AI building blocks for mobile apps
Domains: Image Recognition, Pose Estimation,
Text Recognition

Lowest level: Convert existing models for
HW acceleration and heterogenous scheduling
Android Studio + NDK native libraries with C++ (JNI)

Cloud	**HUAWEI HiAI Service**
Device	**HUAWEI HiAI Engine**
Chip	**HUAWEI HiAI Foundation**

Conclusions



- **Deep Learning is an important workload**
  → Building custom Hardware makes sense

- **Efficient Inference is critical for application**
  - Energy efficiency
  - Low costs
  - Low latency

- **Up to now: DL Accelerators = Matrix-Multiplication Accelerator**
- **Future: Accelerator for Sparse Data and DNN, near-data and in-memory processing**
- **Software (Framework support) and Tooling equally important**

Contact: **Rene Mueller <rene.mueller@huawei.ch>**

HUAWEI

ZURICH RESEARCH CENTER

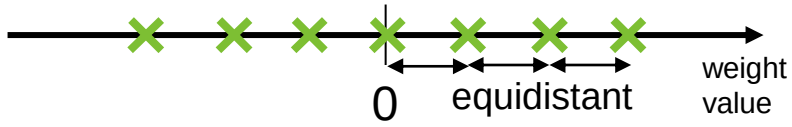
Copyright © 2020 Huawei Technologies Switzerland AG. All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

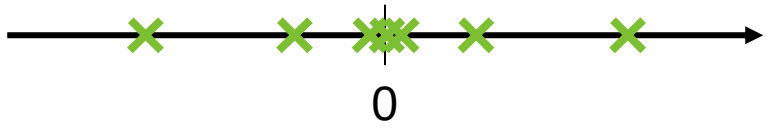
Logarithmic Number Systems

- **Weights are centered around zero.**
- **Numbers around zero should have good representation.**

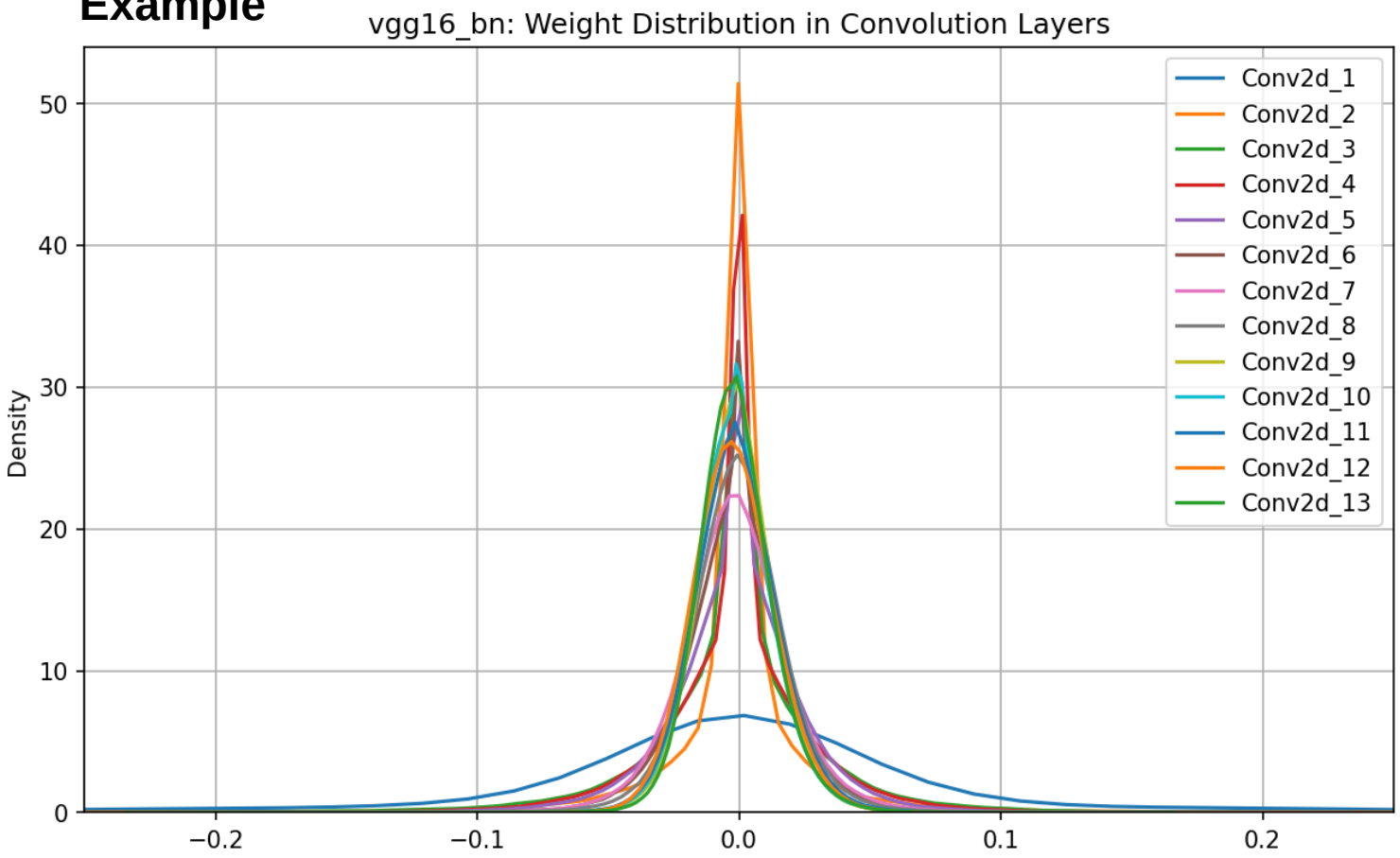
Linear Quantiziation



Logarithmic Quantiziation



Example



[11] Schwartzlander, Alexopoulos: The Sign/Logarithm Number System. In IEEE Trans. of Computers, Vol. C-24, Issue 12, 1975.

Logarithmic Number Systems

$$v = (-1)^s 2^{e.f}$$

Log4.3

1	4	3
s	e	f
0	1110	101

$= 2^{-1.375} \approx 0.385552706352\ldots$

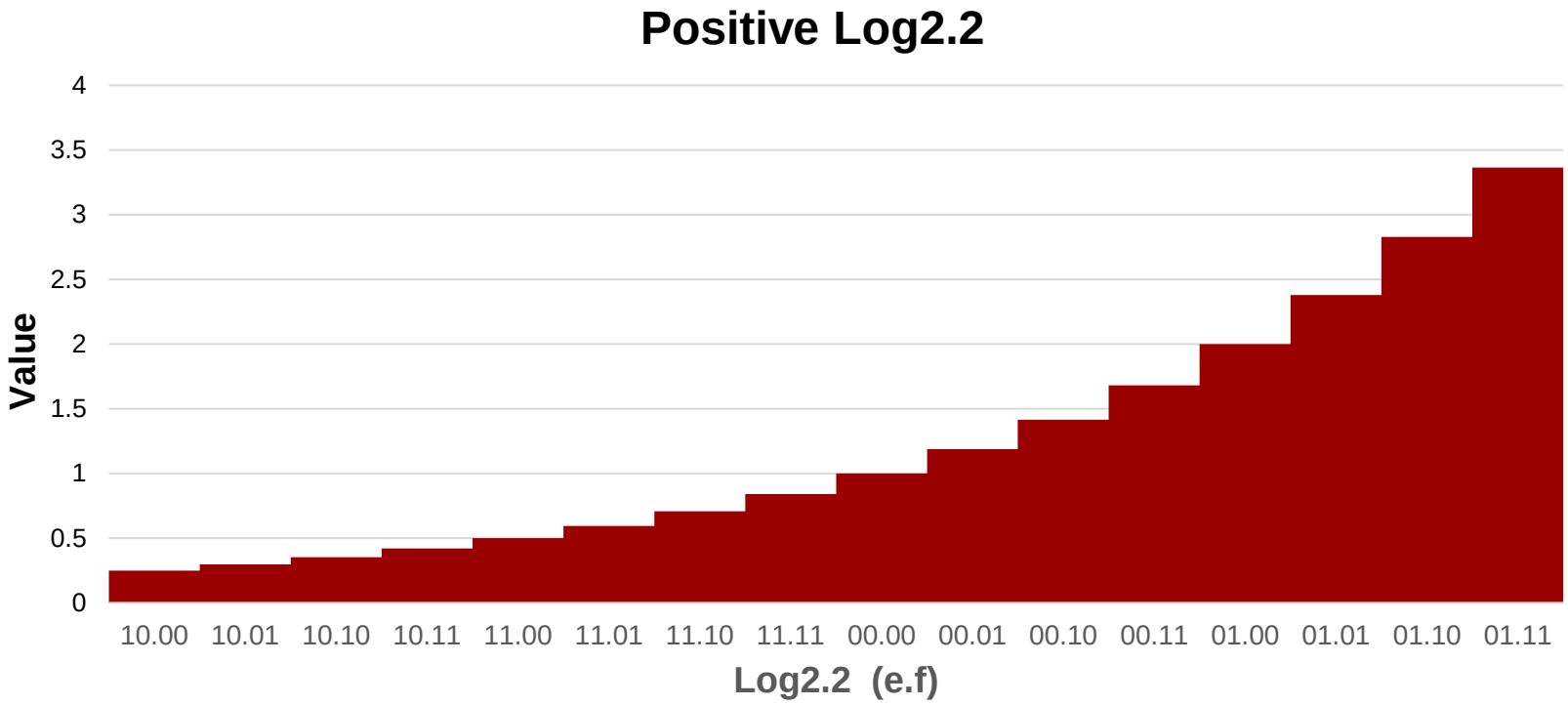
$= (1110.101)_2 = -(0001.011)_2 = -(1 + 1/4 + 1/8)$

$= -1.375$

Example

Log2.2: $v = (-1^s)2^{b_1 b_0 \,.\, b_{-1} b_{-2}}$

($e.f$ without offset)



HUAWEI

Logarithmic Number Systems

- **Representation:** $(-\mathbf{1})^{s} \times \mathbf{2}^{e.f}$
- **Special treatment for zero** (and special encoding)
- **Arithmetic**
  - Multiplication and Division: $\log(x \times y) = \log(x) + \log(y)$, $\log(x/y) = \log(x) - \log(y)$
  - Addition and Subtraction:

$$\log(x \pm y) = \log\left(x\left(1 \pm \frac{x}{y}\right)\right) = \log(x) + \log\left(1 + \frac{x}{y}\right) = \log(x) + \log(1 + 2^{e.f_x - e.f_y})$$

  - Addition/Subtraction requires lookup tables (LUTs) implemented as ROM for linear-to-log conversions

HUAWEI

Von Neumann Laboratory

Huawei Research Center, Zurich

Huawei R&D in Europe

Close to industrial and academic partners, accelerate ICT-based innovations of future life and future work.

Industry Solution Development

- Telecom
- IT
- ITS / Mobility
- Industry 4.0
- IoT
- E-/M-Health

+

Key Technical Innovation

- Cloud and Big Data
- Wireless Networks
- Optical System
- Terminals and IoTings

+

Fundamental Research

- New Theories
- New Materials
- New Components
- New Devices

Huawei R&D in Europe: Open and Joint Value Creation

1,600
industry partners

TO DEVELOP JOINT

Innovative Solutions

230 technical collaboration projects

200+ European technical experts

180+ European Academics

63M € + investment from Huawei in technical partnerships with over 100 of European top universities, research institutes and enterprises

HUAWEI